НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-241
Караульний Максим Миколайович

Перевірив: ас. Баранов О.М.

Одеса – 2026

Звіт з лабораторної роботи №1

Завдання 1. Фактичне питання та задача на міркування

Фактичне питання | *Показник* | *qwen3:4b* | *llama3.2:3b* | |----------|----------|----------|----------| | `response` | Київ | Київ. | | `eval_count` | 346 | 5 | | Час, с | 23.88 | 80.51 | | Токенів/с | 15.66 | 33.77 |

Задача на міркування (середнє з 5 запусків) | *Показник* | *qwen3:4b* | *llama3.2:3b* | |----------|----------|----------|----------| | Правильних із 5 | 1 (з 1) | 0 (з 1) | | `eval_count` | 1608 | 256 | | Час, с | 129.93 | 14.03 |

Питання до чат-бота (Крок 6): 1. **Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однакову, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?**
Відповідь: Усі додаткові токени були витрачені на генерацію внутрішнього «ланцюжка думок» (Chain of Thought), який записується в поле `thinking`. Модель qwen3:4b перед наданням фінальної відповіді розгорнула докладну чернетку самоперевірки, зіставляючи можливі міста (Київ, Львів, Харків тощо) і валідуючи географічні дані. Для користувача кінцевий результат у полі `response` виявився однаковим, проте міркувальна модель витратила значно більше часу й токенів на підготовчий аналіз.

1. **На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?**
   Відповідь: Міркувальні моделі мають чітку перевагу в завданнях, які вимагають багатоетапного планування, математичних розрахунків, аналізу складного коду, вирішення логічних парадоксів чи роботи з нетривіальними умовами, де відповідь залежить від кількох послідовних кроків. Навпаки, їх використання абсолютно невиправдане для простих фактичних запитів, класифікації текстів, перекладу та видобування даних за фіксованим шаблоном, де прихований процес міркування лише створює надмірну затримку (latency) без жодного підвищення точності.
2. **Чим відрізняється процес навчання міркувальної моделі від інструктивної?**
   Відповідь: Інструктивні моделі навчаються методом Supervised Fine-Tuning (SFT) на зразках «запит — пряма відповідь» з наступним вирівнюванням через RLHF/DPO за критеріями корисності та безпеки. Міркувальні моделі навчають на спеціальних масивах даних із розгорнутими покроковими розв'язаннями (CoT) і застосовують великомасштабне навчання з підкріпленням (наприклад, RL з детермінованими верифікаторами правильності результату), що стимулює модель самостійно шукати логічні ланцюжки, здійснювати самоперевірку та виправляти знайдені проміжні помилки.
3. **Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?**
   Відповідь: Ні, більша кількість токенів свідчить лише про більший обсяг обчислень на

етапі інференсу (test-time compute) та багатослівність процесу формування чернетки. Хоча наявність контексту роздумів допомагає моделі розв'язувати складніші логічні зв'язки, вона все одно може галюцинувати або зациклюватися на помилкових вихідних припущеннях. Загальний інтелект моделі лімітується якістю її архітектури, розміром параметрів і глибиною знань, закладених на етапі попереднього навчання.

4. **Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?**
   Відповідь: Швидкість генерації (tokens/s) напряму залежить від архітектури та розміру моделі: `llama3.2:3b` має 3 мільярди параметрів, тоді як `qwen3:4b` — 4 мільярди. Менша кількість параметрів у llama3.2 означає менший обсяг даних, який процесор повинен зчитувати з оперативної пам'яті для кожного наступного токена, завдяки чому llama3.2 генерує помітно швидше (33.77 токенів/с проти 15.66 токенів/с у qwen3:4b на аналогічному обладнанні).

Власні думки: Міркувальні моделі ідеальні для складних задач, але на простих питаннях вони лише марнують ресурси процесора та час очікування.

Завдання 2. Вплив system-повідомлення

Вплив system-повідомлення | *Модель* | *System* | *Довжина відповіді* | *Мова відповіді* | *Кількість речень* | *Інструкцію виконано?* | |-------------|-----------------------|---------------------|-----------------|--------------------|-----------------------| | llama3.2:3b | лаконічний асистент | 2 слова | укр | 1 | Ні (відповіла 2 словами) | | llama3.2:3b | українською, 2 речення | 34 слова | укр | 2 | Так | | qwen3:4b | лаконічний асистент | 1 слово | укр | 1 | Так (але з помилкою у факті) | | qwen3:4b | українською, 2 речення | 30 слів | укр | 2 | Так |

Витрати токенів | *Модель* | *System* | `prompt_tokens` | `completion_tokens` | `total_tokens` | |-------------|-----------------------|-----------------|--------------------|----------------| | llama3.2:3b | лаконічний асистент | 51 | 9 | 60 | | llama3.2:3b | українською, 2 речення | 60 | 87 | 147 | | qwen3:4b | лаконічний асистент | 75 | 1782 | 1857 | | qwen3:4b | українською, 2 речення | 51 | 2790 | 2841 |

Відповіді на питання (Крок 6): 1. **Яка модель точніше дотрималась обмеження «максимум двома реченнями»?**
Відповідь: Обидві моделі успішно дотрималися обмеження і згенерували рівно по два речення, проте `llama3.2:3b` продемонструвала більш природний поділ думки на речення без зайвої фрагментації. 2. **Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?**
Відповідь: Прямої повної відмови від української мови не було зафіксовано, однак малі моделі (зокрема `llama3.2:3b`) схильні до русизмів, калькування та періодичного вживання невластивих українській граматиці слів при складніших конструкціях. 3. **Як змінився prompt_tokens при зміні system-повідомлення і чому?**
Відповідь: Значення `prompt_tokens` зросло (для llama3.2 з 51 до 60 токенів), оскільки системне повідомлення «Відповідай лише українською, максимум двома реченнями.» довше за «Ти лаконічний технічний асистент.», і кожен символ інструкції токенізується та додається до загального контексту. 4. **Чому в qwen3:4b значення completion_tokens значно більше, ніж у**

llama3.2:3b, навіть при однаковій довжині видимої відповіді?
Відповідь: Тому що `qwen3:4b` є міркувальною моделлю, і показник `completion_tokens` підсумовує як видиму користувачеві кінцеву відповідь, так і всі згенеровані приховані токени чернетки (`thinking`), яких виявилося понад дві тисячі.

Питання до чат-бота (Крок 7): 1. **Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?**
Відповідь: Різниця полягає у фокусі тонкого налаштування (fine-tuning) та обсягах навчальних вибірок. Інструктивні моделі натреновані жорстко слідувати директивам формату в запиті користувача, тоді як міркувальні моделі під час генерації роздумів можуть зміщувати свою увагу (attention) на аналіз суті задачі, іноді втрачаючи точний контроль над другорядними обмеженнями форми. 2. **Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?**
Відповідь: Архітектурно для трансформера system і user є послідовністю токенів у контексті, але на етапі навчання (instruction tuning) моделі прищеплюють високий пріоритет до системного повідомлення за допомогою спеціальних службових тегів ролей (`<|im_start|>system`). Модель сприймає системний промпт як глобальний мета-контекст і правила гри, тоді як повідомлення користувача розглядається як локальне поточне завдання. 3. **Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?**
Відповідь: Міркувальна модель формує довгий внутрішній монолог самоперевірки (CoT) перед видачею фінального результату. Токенізатор і API реєструють усі згенеровані моделлю вихідні токени (як приховані, так і фінальні) у загальну метрику `completion_tokens`, навіть якщо інтерфейс відображає лише завершальний абзац. 4. **Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?**
Відповідь: Поведінка залежить від конкретної моделі та її навчання, проте найчастіше модель або намагається знайти компроміс, або віддає перевагу системному повідомленню як пріоритетному фільтру. Проте в невеликих моделях (3B-4B) свіжіше user-повідомлення через механізм близькості в увазі (recency bias) може «перебити» системну вказівку (jailbreak або prompt injection). 5. **Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?**
Відповідь: Малі моделі мають обмежену кількість параметрів та значно бідніше внутрішнє представлення малоресурсних мов (зокрема української) у векторному просторі. Коли складності запиту не вистачає для утримання мовної маски, модель автоматично звалюється в більш потужні статистичні патерни своєї навчальної бази (англійську або російську мову).

Власні думки: System-повідомлення є потужним інструментом, але малі моделі не завжди ідеально його дотримуються. Для суворих форматів однієї інструкції часто замало.

Завдання 3. Порівняння локальної та хмарної моделі

Порівняння локальної та хмарної моделі | *Показник* | *Локальна* | *Хмарна* | |----------|----------|----------| | Назва моделі | llama3.2:3b | gemini-3.5-flash-lite | | Час відповіді, с | 13.29

| 1.27 | | `completion_tokens` | 82 | 221 | | Довжина відповіді, символів | ~260 | ~500 | | Мова відповіді | суржик/укр | укр | | Фактичних помилок у відповіді | 2 | 0 |

Відповіді на питання (Крок 5): 1. **Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?**
Відповідь: У файлі виконання `compare.py` не довелося змінювати жодного рядка, оскільки використано уніфікований інтерфейс клієнта `openai`. Зміни торкнулися лише кількох рядків конфігураційного словника `PROVIDERS` (адреса сервера `base_url`, назва моделі та передача API-ключа). 2. **Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.**
Відповідь: Хмарна модель `gemini-3.5-flash-lite` дала значно повнішу, логічно структуровану та вичерпну відповідь. Вона чітко розмежувала mutability/immutability списків та кортежів, вказала синтаксис (квадратні дужки проти круглих) і детально описала різницю в оптимізації пам'яті, тоді як локальна модель обмежилася загальними коструватими фразами. 3. **Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?**
Відповідь: Так, локальна модель припустилася лексичних русизмів («списки можуть бути змінені», неточне використання терміна «незмінні типи») та частково перекрутила опис продуктивності кортежів. 4. **Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?**
Відповідь: Хмарна модель розгорнута на спеціалізованих серверних прискорювачах (TPU/GPU кластерах), які обробляють запити зі швидкістю сотень токенів за секунду. Ця швидкість генерації на порядки перекриває мережеву затримку (RTT ~100–300 мс), тоді як домашній ПК запускав локальну модель на центральному процесорі.

Питання до чат-бота (Крок 6): 1. **Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?**
Відповідь: OpenAI першими масово випустили зрілий комерційний API, навколо якого індустрія швидко розробила бібліотеки, плагіни та фреймворки (LangChain, LlamaIndex тощо). Щоб розробники могли безболісно мігрувати свої продукти без переписування кодової бази, альтернативні платформи та відкриті проєкти (включно з Ollama, vLLM, Groq) реалізували сумісні специфікації кінцевих точок `/v1/chat/completions`. 2. **Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?**
Відповідь: Розмір моделі (кількість ваг) є визначальним фактором для глибини ерудиції, розуміння складного контексту, володіння синтаксисом малопоширених мов та відсутності фактичних галюцинацій. Проте архітектура, якість очищення датасетів попереднього навчання та ефективність вирівнювання (RLHF) також мають колосальне значення — якісна сучасна 8B-модель сьогодні може перевершувати застарілі 70B-моделі перших поколінь. 3. **Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?**
Відповідь: Високопродуктивні дата-центри використовують високошвидкісну HBM-пам'ять із пропускною здатністю у кілька терабайтів на секунду та тензорні прискорювачі, паралелізовані на рівні тензорного та конвеєрного паралелізму. Звичайний процесор комп'ютера з класичною оперативною пам'яттю DDR4/DDR5 суттєво обмежений швидкістю шини пам'яті, що робить локальний інференс повільнішим навіть за відсутності мережевих затримок. 4. **Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних**

проєктах?
Відповідь: Головний ризик — потенційний витік конфіденційних даних клієнтів, інтелектуальної власності чи комерційної таємниці на сервери стороннього провайдера, а також ризик використання запитів для подальшого тренування майбутніх моделей. У комерційному секторі укладають Enterprise-угоди (BAA, DPA, SLA) із гарантіями Zero Data Retention (ZDR), використовують хмарні анклави (наприклад, Azure OpenAI, AWS Bedrock) або встановлюють локальні відкриті моделі на власних серверах компанії. 5. **Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?**
Відповідь: Rate limit — це встановлене провайдером обмеження на кількість запитів (RPM — requests per minute) та токенів (TPM — tokens per minute), призначене для захисту інфраструктури від перевантажень і зловживань. Розробники обходять або згладжують ці ліміти за допомогою черг запитів, експоненційного відкату при повторних спробах (exponential backoff з ретраями), локального кешування відповідей (Redis) та балансування навантаження між декількома ключами/провайдерами.

Власні думки: Локальну варто обирати для роботи з конфіденційними даними (наприклад, медичними), а хмарну — для всього іншого, де потрібна швидкість і високий інтелект.

Завдання 4. Вимірювання швидкості

Результати вимірювання швидкості | *Розміщення* | *Модель* | *Промпт* | *TTFT, с* | *Всього, с* | *Симв./с* | |--------------|----------|----------|-----------|-------------|-----------| | локальна | llama3.2:3b | короткий | 2.17 | 2.72 | 16.6 | | локальна | llama3.2:3b | середній | 2.17 | 4.66 | 62.3 | | локальна | llama3.2:3b | довгий | 2.22 | 9.60 | 88.5 | | хмарна | gemini-3.5-flash-lite | короткий | 0.60 | 0.64 | 37.7 | | хмарна | gemini-3.5-flash-lite | середній | 0.70 | 1.20 | 380.5 | | хмарна | gemini-3.5-flash-lite | довгий | 0.71 | 3.13 | 877.3 |

Холодний і теплий старт локальної моделі | *Показник* | *Холодний старт* | *Теплий старт* | |------------------|----------------|----------------| | TTFT, с | ~35.07 | 2.17 | | Всього, с | ~80.51 | 2.72 |

Відповіді на питання (Крок 6): 1. **У якої моделі кращий TTFT і чому саме в неї?**
Відповідь: За стабільного теплого підключення кращий TTFT продемонструвала хмарна модель `gemini-3.5-flash-lite` (0.60 с проти 2.17 с у локальної), оскільки хмарний бекенд використовує надпотужні обчислювальні ресурси та кешування контексту, тоді як локальний процесор довше рахує вхідні матричні добутки. 2. **Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.**
Відповідь: У цьому експерименті хмарна модель виявилася швидшою за обома показниками, проте загалом вони не зобов'язані збігатися: модель може мати великий TTFT через довгий префікс або мережеву затримку, але потім видавати сотні символів за секунду на етапі декодування. 3. **Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?**
Відповідь: На короткому промпті різниця у загальному часі відповіді була невеликою (2.72 с проти 0.64 с), проте на довгому промпті перевага хмарної моделі стала колосальною: швидкість генерації досягла 877.3 симв/с проти 88.5 симв/с у локальної, скоротивши час очікування більш ніж утричі. 4. **Наскільки різняться три повтори одного виміру? Чи достатньо було б одного**

запуску?
Відповідь: Повторні виміри коливалися в межах 10–25% через фонове навантаження на систему, коливання інтернет-з'єднання та джитер мережі. Одного запуску категорично недостатньо для надійного бенчмарку, оскільки випадковий сплеск затримки може суттєво спотворити підсумкові висновки. 5. **Чому холодний старт повільніший за теплий?**
Відповідь: Під час холодного старту сервіс Ollama повинен зчитати бінарні файли ваг моделі з накопичувача (SSD/HDD) у системну пам'ять (RAM/VRAM) та ініціалізувати обчислювальний граф, що займає десятки секунд, тоді як теплий старт опрацьовує запит миттєво з уже завантаженої в пам'ять моделі.

Питання до чат-бота (Крок 7): 1. **Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?**
Відповідь: Під час TTFT відбувається повна паралельна обробка всього вхідного промпту (Prompt Processing або етап prefill): токенізація, створення KV-кешу для всіх вхідних токенів та генерація першого вихідного токена. Цей показник вимірюють окремо, оскільки він критично впливає на сприйняття користувача (відчуття чуйності інтерфейсу), тоді як швидкість генерації визначає темп подальшого виводу тексту. 2. **Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?**
Відповідь: Хмарна модель може страждати від фізичної затримки передачі пакетів мережею (мережевий RTT, черги балансувальника та авторизація), що погіршує TTFT. Проте після старту генерації її промислові графічні прискорювачі з величезною пропускною здатністю пам'яті вистрілюють токени набагато швидше, ніж локальний процесор. 3. **Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?**
Відповідь: Холодний старт — це початкове розгортання моделі «з нуля», що включає створення системного процесу, виділення віртуальної пам'яті та зчитування гігабайтів ваг із дискового накопичувача. Після завантаження модель залишається резидентною в пам'яті (warm state) протягом певного тайм-ауту (за замовчуванням 5 хвилин в Ollama), і всі наступні запити виконуються без накладних витрат на читання з диска. 4. **Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?**
Відповідь: Медіана є стійкою (робастною) статистичною метрикою, яка не чутлива до поодиноких екстремальних викидів — таких як затримка збирання сміття в ОС, раптове фонове навантаження від антивірусу або випадковий мережевий таймаут. 5. **Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?**
Відповідь: Вхідний запит обробляється паралельно: механізм самоуваги (self-attention) обчислює матриці для всіх токенів промпту за один масивний матричний прохід. Генерація відповіді є авторегресійною та строго послідовною: кожен новий токен залежить від попереднього, тому вимагає окремого повного прямого проходу моделі та повторного звернення до всієї пам'яті ваг.

Власні думки: Для інтерактивного чат-бота найважливіший TTFT (користувач не любить чекати відповіді), а для фонової обробки документів (наприклад, аналіз тисячі відгуків) — загальна швидкість генерації (симв/с).

Завдання 5. Оцінювання якості за рубрикою

Оцінка якості за рубрикою | *Завдання* | *llama3.2:3b* | *qwen3:4b* | *Хмарна модель* | |---------------|---------------|------------|-----------------| | 1. Факт | 0 | 2 | | 2. Формат | 1 | 1 | 2 | | 3. Стиснення | 0 | 2 | 2 | | 4. Код | 1 | 1 | 2 | | 5. Логіка | 1 | 2 | 1 | | 6. Українська | 0 | 1 | 2 | | *Разом /12* | 3 | 9 | 11 |

Розбіжність оцінок між двома прогонами (так / ні) | *Завдання* | *llama3.2:3b* | *qwen3:4b* | *Хмарна модель* | |---------------|---------------|------------|-----------------| | 1. Факт | так | ні | ні | | 2. Формат | так | ні | ні | | 3. Стиснення | так | ні | ні | | 4. Код | так | ні | ні | | 5. Логіка | так | ні | так | | 6. Українська | так | ні | ні |

Відповіді на питання (Крок 6): 1. **Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?**
Відповідь: Найбільше балів набрала хмарна модель `gemini-3.5-flash-lite` (11 з 12), а серед локальних упевнену перевагу продемонструвала міркувальна `qwen3:4b` (9 з 12) проти слабкої інструктивної `llama3.2:3b` (3 з 12). Основна різниця сформувалася на завданнях логічного міркування (де qwen безпомилково застосував формулу включень-виключень), збереженні чисел під час стиснення тексту та загальній якості української мови. 2. **На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?**
Відповідь: У моделі `llama3.2:3b` оцінки розійшлися майже на всіх завданнях через високу дисперсію та схильність до галюцинацій при повторних спробах, що свідчить про вкрай низьку надійність. У `gemini-3.5-flash-lite` розбіжність сталася лише на логічній задачі (в першому прогоні модель помилилася в арифметиці $12+9-4=21$, а в другому розв'язала вірно). Модель `qwen3:4b` показала ідеальну стабільність між прогонами, отримавши однакові результати в обох спробах. 3. **Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?**
Відповідь: Лише хмарна модель у першому запуску надала чистий JSON, тоді як обидві локальні моделі (`llama3.2:3b` та `qwen3:4b`) стабільно обгортали відповідь у markdown-блоки ```` ```json ... ``` ````, що призводить до збою стандартного `json.loads()` без попередньої очистки рядка. 4. **Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?**
Відповідь: У `llama3.2:3b` спостерігалися критичні проблеми: повна галюцинація незв'язного змісту (пояснення стека як «морського рака» або «горизонтальної геологічної породи») та грубі русизми («поскольку», «горної порід»). Модель `qwen3:4b` продемонструвала набагато грамотнішу українську мову з коректною термінологією, але зі стилістичними кальками, тоді як хмарна модель сформувала бездоганний літературний текст.

Приклад провалу (Крок 7): *Запит:* `Надай інформацію про Одеський національний університет імені І. І. Мечникова у форматі JSON: name, year, city. Без пояснень, без markdown, лише JSON.`
Відповідь моделі (qwen3:4b):

```
{
  "name": "Одеський національний університет імені І. І. Мечникова",
  "year": 1865,
  "city": "Одеса"
}
```

Яка правильна відповідь мала бути:
Мав бути чистий машинозчитуваний текст JSON без оформлення кодовими блоками:
`{"name": "Одеський національний університет імені І. І. Мечникова", "year": 1865, "city": "Одеса"}`
Пояснення: Модель проігнорувала негативну інструкцію «без markdown», оскільки під час навчання (fine-tuning) на датасетах програмування патерн огортання будь-якого структурованого коду та JSON у блок із трьох бектіків закріпився як домінантний, що переважає над системними промптами звичайної складності.

Питання до чат-бота (Крок 8): 1. **Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?**
Відповідь: В індустрії застосовують стандартизовані бенчмарки з автоматичним метричним оцінюванням (MMLU для загальної ерудиції, GSM8K та MATH для математики, HumanEval для коду), підхід LLM-as-a-Judge (використання топових моделей на кшталт GPT-4o для перевірки відповідей за рубриками), краудсорсингові платформи сліпого A/B-тестування (LMSYS Chatbot Arena) та модульне тестування конкретних сценаріїв у розробці агентів через фреймворки на зразок Ragas чи DeepEval. 2. **Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?**
Відповідь: Масиви коду в інтернет-корпусах (GitHub, StackOverflow, документація) практично завжди відформатовані всередині markdown-блоків. Цей статистичний пріоритет настільки сильний, що проста негативна текстова вказівка («без markdown») часто програє механізму автодоповнення токенів; надійне вирішення досягається лише через вимушене декодування (Structured Outputs / JSON mode / граматики BNF на рівні інференс-рушія). 3. **Чому малі мовні моделі гірше пишуть українською, ніж англійською?**
Відповідь: Англійська мова становить понад 80–90% більшості навчальних корпусів LLM, тоді як частка україномовного сегмента зазвичай менша за 0.5%. Малим моделям із ємністю 3–4 млрд параметрів елементарно не вистачає нейронної місткості для детального закріплення складної словозмінної граматики малоресурсних мов, тому вони використовують англійські концепти й перекладають їх з русизмами або граматичними помилками. 4. **Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?**
Відповідь: При ненульовій температурі генерація є ймовірнісним процесом (stochastic sampling): модель не обирає фіксований результат, а вибирає токени із розподілу ймовірностей. Якщо на ранньому етапі випадково обирається альтернативний токен, уся подальша траєкторія уваги трансформера кардинально змінюється, що може призвести як до геніального інсайту, так і до критичної галюцинації. 5. **Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?**
Відповідь: Галюцинація — це генерація семантично правдоподібного, граматично правильного, але фактично хибного тексту. Оскільки LLM є авторегресійними предикторами наступного токена, а не базами знань із перевіркою істинності, вони оптимізовані максимізувати гладкість тексту, а фраза «я не знаю» зустрічається в навчальних даних значно рідше за впевнене продовження твердження.

Власні думки: Оцінка за рубрикою є надійним інженерним інструментом, що дозволяє об'єктивно розкрити сильні та слабкі сторони архітектур. Найпоказовішим виявилося завдання на

логіку (де міркувальна `qwen3:4b` із результатом 2 бали випередила як `llama3.2:3b`, так і перший прогін `gemini`), а для роботи з українською мовою хмарна модель наразі є безальтернативним лідером за якістю.

Завдання 6. Керування випадковістю (temperature і top_p)

Вплив параметрів на різноманітність відповідей | *Параметр* | *Значення* | *Унікальних із 5 (творче)* | *Правильних із 5 (фактичне)* | |---|---|---|---| | temperature | 0.0 | 2 | 0 | | temperature | 0.7 | 5 | - | | temperature | 1.5 | 5 | 0 | | top_p | 0.1 | 1 | - | | top_p | 0.5 | 5 | - | | top_p | 1.0 | 5 | - |

Перевірка відтворюваності | *Показник* | *Значення* | |---|---|---| Кількість запусків | 10 | | temperature | 0.0 | | Унікальних відповідей | 1 | | Усі відповіді однакові (так / ні) | так |

Відповіді на питання (Крок 7): 1. **Яке значення температури ви поставите агенту, який має повертати JSON, і чому?**
Відповідь: Для виведення JSON слід встановлювати `temperature=0.0`. Це гарантує максимальну детермінованість, строге дотримання схеми даних і запобігає випадковим галюцинаціям у синтаксисі, які ламають роботу парсера. 2. **Яке — генератору варіантів заголовків?**
Відповідь: Для генерації заголовків або слоганів оптимальною є температура в діапазоні `0.7–1.0`. Це забезпечує креативність, багатство синонімів та високу різноманітність ідей без ризику скочування у повне безглуздя. 3. **Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?**
Відповідь: `temperature` масштабує весь розподіл ймовірностей (робить його більш плоским або загостреним), змінюючи відносні шанси всіх слів, тоді як `top_p` (nucleus sampling) динамічно відсікає маловрогідний «хвіст» розподілу, розглядаючи лише найімовірніші токени із сумарною часткою p, залишаючи їх внутрішні пропорції незмінними. 4. **Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?**
Відповідь: У проведеному експерименті всі 10 відповідей виявилися абсолютно однаковими (100% повторюваність). Проте в загальному випадку повної гарантії немає через неасоціативність обчислень із рухомою комою (floating-point nondeterminism) при багатопотоковому виконанні на GPU. 5. **Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.**
Відповідь: Модель видала: «Рік заснування Одеська Політехніка - 2045». При надвисокій температурі ($T=1.5$) ймовірності навіть найбільш малоймовірних токенів підтягуються до лідерів, через що модель випадково обирає випадкові числа з контексту майбутнього, втрачаючи причинно-наслідковий зв'язок.

Питання до чат-бота (Крок 8): 1. **Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?**
Відповідь: Математично параметр температури ділить сирі вихідні логіти z_i моделі перед застосуванням функції softmax: $P(y_i) = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$. При $T \to 0$ розподіл вироджується в дельта-функцію (argmax), а при зростанні $T > 1$ він згладжується, наближаючись до рівномірного розподілу. 2. **Чим top_p відрізняється від temperature —**

обидва впливають на різноманітність, але за яким механізмом?
Відповідь: `temperature` діє глобально на форму всього розподілу, тоді як `top_p` є фільтром урізання простору пошуку (nucleus sampling). Він сортує токени за спаданням і відсікає всі варіанти, чия сукупна накопичена ймовірність перевищує поріг p, захищаючи генерацію від вибору катастрофічно невідповідних слів навіть при помірній температурі. 3. **Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?**
Відповідь: Більшість фахівців та інструкцій провайдерів (включно з OpenAI) рекомендують фіксувати один параметр і змінювати інший (наприклад, залишати `top_p=1.0` і регулювати лише `temperature`). Їх одночасна зміна ускладнює прогнозування поведінки моделі та робить налаштування надлишковим. 4. **Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?**
Відповідь: Головна причина полягає в апаратному паралелізмі сучасних GPU/TPU. Операції паралельної редукції та матричного множення чисел із плаваючою точкою (float16/bfloat16) не мають властивості асоціативності: порядок додавання елементів змінюється між запусками потоків, мікроскопічна похибка накопичується і на критичних токенах-кандидатах з майже рівними логітами argmax може обрати інший токен. 5. **Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?**
Відповідь: Для вилучення сутностей, написання коду, парсингу JSON та аналітичних відповідей використовують $T = 0.0–0.2$. Для відповідей асистента в розмовних сценаріях — $T = 0.6–0.8$. Для творчого копірайтингу, маркетингових слоганів та генерації нестандартних ідей — $T = 1.0–1.2$.

Власні думки: Для отримання структурованих даних (JSON) в наступних лабораторних обов'язково треба ставити temperature=0, інакше агент колись обов'язково зламається через випадкову галюцинацію у синтаксисі. Налаштування "за замовчуванням" (зазвичай 0.7) абсолютно не підходить для програмної інтеграції.

Підсумкове порівняння моделей та Висновок

Підсумкове порівняння моделей | *Критерій* | *Локальна мала (3B)* | *Локальна більша (4B/8B)* | *Хмарна* | |----------|----------|----------|----------| | Швидкість генерації | Середня (~30-35 ток/с) | Низька (~15-20 ток/с) | Дуже висока (~800+ симв/с) | | TTFT | Добра (~2 с на теплому) | Задовільна (~2-3 с) | Найкраща (~0.6-0.7 с) | | Якість, балів /12 | 3 | 9 | 11 | | Стабільність формату (завдання 2) | Низька | Середня | Висока | | Потрібні ресурси | ~3 ГБ VRAM / RAM | ~5 ГБ VRAM / RAM | Тільки інтернет | | Вартість | Безкоштовно | Безкоштовно | Платна (або безкоштовні ліміти) | | Куди йдуть ваші дані | Нікуди (локально) | Нікуди (локально) | На сервери провайдера | | Працює без інтернету | Так | Так | Ні | | Коли обирати саме її | Для простих завдань на слабкому ПК офлайн | Для логічних/аналітичних задач локально без інтернету | Для високої якості, швидкості та складних агентів |

Висновок: Під час виконання лабораторної роботи я розгорнув локальне середовище з використанням Ollama та порівняв інструктивну (`llama3.2:3b`) і міркувальну (`qwen3:4b`) моделі. Також я підключив хмарну модель через OpenAI-сумісний API. Я дослідив вплив системних

промптів на поведінку моделей, а також проаналізував залежність якості генерації від параметрів `temperature` і `top_p`.

Головне правило вибору: локальну модель слід обирати для роботи з приватною інформацією (комерційною таємницею) або коли немає доступу до інтернету, попри її меншу загальну ерудицію. Хмарну модель слід обирати в усіх інших випадках, оскільки вона забезпечує значно вищу надійність збереження формату (JSON), краще володіння українською мовою та вищу швидкість генерації довгих текстів.

Додатково варто відзначити, що робота з різними параметрами генерації (на кшталт `temperature` та `top_p`) наочно продемонструвала баланс між креативністю та точністю. Для суворих інженерних задач (наприклад, генерація формату JSON) критично важливо зводити випадковість до нуля (`temperature=0`), тоді як для творчих завдань підвищення температури дозволяє моделі розкрити свій потенціал. Загалом, практичний досвід розгортання локальних моделей підтвердив їхню життєздатність як інструменту для автономних та захищених систем, хоча для загальних завдань хмарні рішення наразі залишаються лідерами за швидкістю та якістю відповідей.